

Redis — Practical System Design Notes

Hinglish • Common-sense • Interview-focused • Core concepts + Q&A;

1. Redis Actually Kya Hai?

Redis ek fast in-memory data store hai. Simple mental model: Database main store room hai; Redis fast-access shelf hai. Frequently needed data ko RAM mein rakhkar application ko faster reads/writes mil sakte hain.

```
Application
↓
Redis (RAM-oriented, very fast)
↓ cache miss
Database (durable source of truth)
```

Redis sirf cache nahi hai. It can also be used for sessions, counters, rate limiting, queues/streams, distributed coordination and other data-structure-based workloads.

2. Why Redis?

Database ko har request par hit karna expensive/slow ho sakta hai, especially repeated reads ke case mein. Redis hot data ko memory mein rakhkar database load aur latency reduce kar sakta hai.

```
Without Redis:
User → Node.js → DB → Product
User → Node.js → DB → Product
User → Node.js → DB → Product

With Redis:
User → Node.js → Redis → Product (HIT)
                        ↓
                        DB      (only when needed)
```

Interview question: Redis use karne ka primary reason kya hai?

Frequently accessed or shared data ko very fast access dena, database load reduce karna, and suitable use cases mein low-latency coordination/state handling.

3. Redis RAM Mein Hai — Restart Ke Baad Kya?

Redis primarily memory-based hai, but it supports persistence mechanisms such as RDB snapshots and AOF. Isliye 'Redis restart hua = data hamesha lost' incorrect hai. Actual durability configured persistence, replication, deployment, and failure scenario par depend karti hai.

```
RAM
↓
Redis data
```

```
Persistence options:
RDB → periodic snapshots
AOF → append operations to a log
```

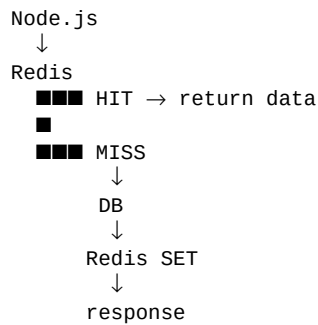
```
Replication / managed Redis
→ additional availability options
```

Mental model: Redis fast memory store hai, but it can persist data. However, application ke critical durable source of truth ko Redis par blindly depend nahi karna chahiye unless architecture explicitly designed for it.

4. Cache-Aside Pattern

Most common caching pattern: application pehle Redis check karti hai. HIT ho toh data return. MISS ho toh DB se fetch, Redis mein cache, then response.

```
GET /product/123
```



Question: Cache miss par kya hota hai?

DB/source se data fetch hota hai, cache mein suitable TTL ke saath store kiya ja sakta hai, then client ko response diya jata hai.

5. TTL / Expiration

Redis key ko expiry de sakte ho. Example: OTP, temporary cache, session data, rate-limit counters.

```
SET otp:user123 4821 EX 300
```

Meaning:

key = otp:user123

value = 4821

expire = 300 seconds

After expiry:

key automatically becomes unavailable.

TTL stale data ko indefinitely memory mein rakhne se bachata hai. TTL choose karna data freshness aur cache-hit rate ka trade-off hai.

6. Cache Invalidation

Classic problem: DB mein product price change ho gayi, but Redis mein old price cached hai. Cache invalidation strategy decide karti hai ki stale value kab remove/update hogi.

DB:
product 123 → ■600

Redis:
product:123 → ■500 ← stale

Possible strategies:

1. Delete cache after DB update
2. Update cache after DB update
3. Short TTL as fallback
4. Versioning / event-driven invalidation

Common pattern: write to DB, then invalidate or update the relevant cache key. Exact ordering and consistency guarantees depend on the application.

7. Cache Stampede / Thundering Herd

Suppose ek popular key expire ho gayi aur simultaneously 10,000 requests aa gayi. Sab Redis miss dekhkar DB ko hit kar dein, toh DB par sudden spike aa sakta hai.

10,000 requests
↓
Redis MISS
↓
10,000 DB queries ■
↓
DB spike

Mitigations include TTL jitter, request coalescing/single-flight, locks, background refresh, stale-while-revalidate patterns, and careful cache warming depending on the use case.

Question: Cache stampede kya hai?

Jab many requests ek saath missing/expired cached item ko regenerate karne ke liye backend/database par load create karti hain.

8. Redis Data Structures

Type	Simple use case
String	Cache value, counter, token-like data
Hash	Object-like fields, e.g. user profile fields
List	Ordered collection / simple queue patterns
Set	Unique members
Sorted Set	Leaderboard / ranking by score
Stream	Event/message stream workloads

Important: choose the data structure based on the access pattern, not just because Redis supports it.

9. Redis for Sessions

Multiple Node.js servers ke saath in-memory sessions problematic ho sakti hain because Server 1 ki RAM ka session Server 2 ke paas nahi hota. Redis shared session store provide kar sakta hai.

User
↓
Load Balancer
↓

Server 1 → Redis ← Server 2
 ← Server 3

Session lives in shared Redis,
not only in one Node process.

Alternative is a token-based stateless architecture, depending on requirements. Redis is useful when server-side session state is needed.

Question: Why is Redis useful behind a load balancer?

It provides shared fast state so any healthy application instance can access session/cache/rate-limit information.

10. Redis Rate Limiting

Rate limiter ka simple idea: ek user/IP ko fixed time window mein limited requests allow karna. Redis atomic counters/expiry and suitable algorithms ke liye useful hai.

```
user:123  
requests in current window = 98
```

```
Limit = 100
```

```
98 → allow  
99 → allow  
100 → allow  
101 → reject / 429
```

Common algorithms: fixed window, sliding window, token bucket, leaky bucket. Redis can implement or support these patterns, but the exact algorithm depends on traffic and fairness requirements.

11. Atomic Operations

Redis commands such as INCR are atomic at the command level, which is useful for counters and rate limiting. For multiple operations that must behave as one logical unit, Redis transactions/scripts or appropriate atomic patterns may be needed.

```
INCR rate:user123
```

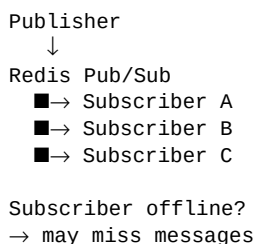
If 100 requests arrive concurrently,
Redis can safely increment the counter
without a simple read-modify-write race
for the INCR operation itself.

Question: Why not do GET counter → increment in Node.js → SET counter?

Concurrent requests can read the same old value and overwrite each other's updates. An atomic increment avoids that lost-update pattern.

12. Redis Pub/Sub vs Persistent Messaging

Redis Pub/Sub is useful for live fan-out where subscribers need current messages, but traditional Pub/Sub is not a durable queue: a subscriber that is disconnected can miss messages. For durable/event-processing requirements, Redis Streams or a dedicated durable messaging system may be more appropriate.

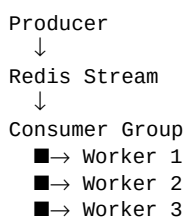


Question: Should you use Pub/Sub for payment events that must never be lost?

Not as a simple fire-and-forget Pub/Sub design. Use a durable messaging pattern with acknowledgements/replay/retention appropriate to the requirement.

13. Redis Streams / Queue-like Workloads

Redis Streams provide an append-oriented stream with consumer groups and acknowledgement patterns, making them more suitable than basic Pub/Sub for many durable-ish event processing workflows. For very large or specialized messaging systems, Kafka/RabbitMQ/SQS and other brokers may be better depending on requirements.



Messages can be tracked/acknowledged according to the stream/consumer design.

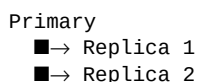
14. Redis Persistence: RDB vs AOF

Mechanism	Simple mental model	Trade-off
RDB	Periodic snapshots	Compact snapshots; recent writes may be absent after a crash depending
AOF	Append write operations	Can provide more granular recovery; more write/log overhead depending

Persistence is a durability feature, not a replacement for backups/replication or a carefully designed database of record.

15. Redis Replication and High Availability

A single Redis instance can become a failure point. Replicas can maintain copies of data, and Redis Sentinel or Redis Cluster can provide different high-availability/scaling models.

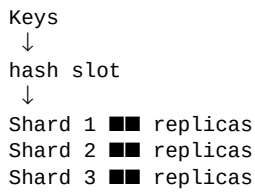


If primary fails:
HA mechanism can promote/re-route according to deployment configuration.

Replication is not the same as backup: replicas can replicate accidental deletes or corruption. Backups and recovery strategy still matter.

16. Redis Cluster

Redis Cluster distributes keys across multiple Redis nodes using sharding. It helps scale memory and throughput horizontally and can provide replicas for shards.



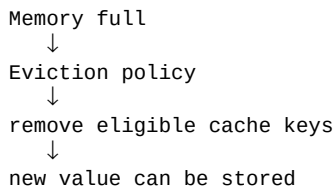
Cluster is not simply 'three Redis servers with the same data'. Sharding means different keys can live on different primary nodes.

Question: Why use Redis Cluster?

To distribute data/memory and workload across multiple Redis nodes and support larger scale than a single instance can provide.

17. Cache Eviction

Redis memory is finite. When memory pressure reaches configured limits, eviction policies determine what cached keys can be removed. Policies may target volatile keys, all keys, LRU/LFU-like strategies, depending on configuration/version.



For cache workloads, eviction policy should match the data's importance and access pattern. Do not treat all Redis data as disposable cache if some keys are critical state.

18. Cache-Aside vs Write-Through vs Write-Behind

Pattern	Simple idea	Typical trade-off
Cache-aside	App reads/writes DB and manages cache	Flexible; invalidation complexity
Write-through	Write cache and underlying store together through a coordinator	Fast reads; more write path complexity
Write-behind	Write cache first and persist later	Fast writes; durability/ordering complexity

For most interview discussions, start with cache-aside unless the problem explicitly requires a different consistency/write pattern.

19. Hot Keys

A hot key is a single Redis key receiving a disproportionate amount of traffic, such as a viral product or a single global counter. Even with many Redis nodes, one hot key can concentrate load.

```
10 million requests
↓
"product:viral"
↓
same Redis key
↓
hot key bottleneck
```

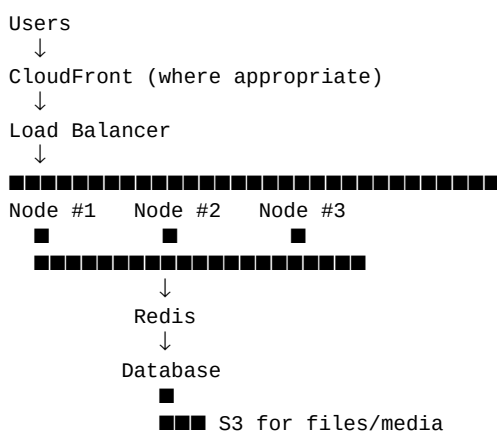
Possible approaches include local caching, key replication/splitting where semantics allow, request coalescing, rate limiting, and architectural redesign.

20. Redis vs Database

Question	Redis	Typical DB
Primary strength	Very low-latency memory operations	Durable structured data / transactions / querying
Typical role	Cache, sessions, counters, coordination, fast state	Source of truth
Persistence	Optional/configurable	Core property
Querying	Data-structure commands	Rich query/index capabilities depending on DB

Do not say 'Redis is a faster database so replace MySQL/Postgres'. Redis and a relational database solve different problems and can complement each other.

21. Real Production Architecture



Redis may be used for cache, sessions, rate limiting, counters, locks/coordination, or streams. It should not automatically become the source of truth for every piece of data.

22. High-Value Interview Questions + Short Answers

Question	Short answer
Why Redis?	Very fast in-memory access; reduce latency and DB load for suitable workloads.
Is Redis only a cache?	No. Cache, sessions, counters, rate limiting, streams, coordination, etc.
Does Redis lose everything on restart?	Not necessarily; persistence can be configured via RDB/AOF, plus replication/HA.
What is cache-aside?	Read Redis → miss → DB → populate Redis → return.
What is TTL?	Automatic expiry time for a key.
What is cache stampede?	Many misses regenerate the same data and overload the backend.
Why Redis for sessions?	Shared low-latency state across multiple app servers.
Why Redis for rate limiting?	Fast counters, expiry and atomic operations.
Why INCR instead of GET+SET?	Atomic increment avoids simple concurrent lost-update races.
Pub/Sub for critical durable events?	Basic Pub/Sub can lose messages for disconnected subscribers; use durable messaging/Streams.
RDB vs AOF?	Snapshots vs append-style operation log; different durability/recovery trade-offs.
Replication vs backup?	Replication copies current state; it does not replace independent backups.
Why Redis Cluster?	Shard data/workload across nodes and scale memory/throughput.
What is a hot key?	One key receiving disproportionate traffic, potentially creating a bottleneck.

23. Final Cheat Sheet

Redis

→ Fast in-memory data store

Cache-aside

→ Redis HIT → return

→ MISS → DB → Redis → return

TTL

→ Automatic key expiry

Invalidation

→ Remove/update stale cache

Stampede

→ Many misses hit DB together

String

→ Simple values/counters

Hash

→ Object-like fields

Set

→ Unique values

Sorted Set

→ Leaderboards/ranking

Stream

→ Event stream / consumer groups

Sessions

→ Shared state across app servers

Rate limiting

→ Atomic counters + expiry / suitable algorithms

RDB

→ Snapshots

AOF

→ Append operation log

Replication

→ Copies data to replicas

Cluster

→ Sharding + distributed Redis

Eviction

→ What to remove when memory is full

Hot key

→ One key gets too much traffic

Golden rule:

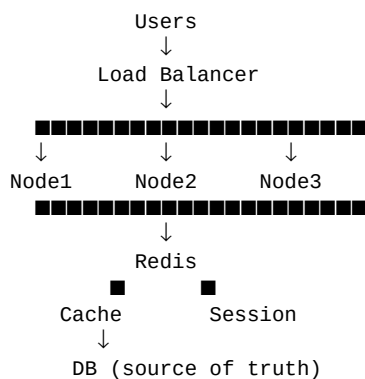
Redis is usually a speed/state layer;
your durable source of truth should be chosen deliberately.

24. Practice Questions

1. 100 Node.js requests hit the same product. How would Redis reduce DB load?
2. What happens when the product key is missing from Redis?
3. Why can a short TTL improve freshness but reduce cache hit rate?
4. What happens if 10,000 requests arrive exactly when a hot key expires?
5. How would you prevent a cache stampede?

6. Why is local in-memory session storage a problem with multiple Node servers?
7. Why is Redis a good fit for rate limiting?
8. Why can GET + increment + SET be unsafe under concurrency?
9. Would you use basic Redis Pub/Sub for a payment event that cannot be lost? Why?
10. What is the difference between Redis replication and backup?
11. When would Redis Cluster become useful?
12. What is a hot key and how could it hurt performance?
13. When would you choose S3 instead of Redis?
14. What should happen if Redis is temporarily unavailable?

25. One Architecture to Remember



S3 + CloudFront
→ separate path for large files/media

Next practical exercise: Run 3 Node.js instances + Redis + one database locally. Implement product GET with cache-aside, TTL, cache invalidation after update, and a simple rate limiter. Then observe the difference in DB calls with and without Redis.